

Student 7 **DUDEKULA CHAND BASHA** 192424218 RD

5 / 5 submitted

Q1. Smart Document Editor using String, StringBuilder and StringBuffer
CODE

```
import java.util.Scanner;
public class SmartDocumentEditor {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String paragraph = sc.nextLine();
        int characters = paragraph.length();
        String[] words = paragraph.trim().split("\\s+");
        int wordCount = words.length;
        String[] sentenceArray = paragraph.split("[.!?]+");
        int sentenceCount = 0;
        for (String s : sentenceArray) {
            if (!s.trim().isEmpty())
                sentenceCount++;
        }
        System.out.println("Document Statistics");
        System.out.println("Characters : " + characters);
        System.out.println("Words      : " + wordCount);
        System.out.println("Sentences  : " + sentenceCount);
        String oldWord = sc.next();
        String newWord = sc.next();
        String replacedParagraph = paragraph.replace(oldWord, newWord);
        StringBuffer history = new StringBuffer();
        history.append("Original Document:\n");
        history.append(paragraph);
        history.append("\n\nReplaced ");
        history.append(oldWord);
        history.append("' with '");
        history.append(newWord);
        history.append("\n");
        StringBuilder finalDocument = new StringBuilder();
        String[] sentences = replacedParagraph.split("(?<=[.!?])");
        for (String sentence : sentences) {
            sentence = sentence.trim();
            if (sentence.length() == 0)
                continue;
            char mark = ' ';
            if (sentence.endsWith(".") || sentence.endsWith("!") || sentence.endsWith("?")) {
                mark = sentence.charAt(sentence.length() - 1);
                sentence = sentence.substring(0, sentence.length() - 1);
            }
            String reversed = new StringBuilder(sentence).reverse().toString();
            finalDocument.append(reversed);
            if (mark != ' ')
                finalDocument.append(mark);
            finalDocument.append(" ");
        }
        history.append("Each sentence reversed.\n");
        System.out.println("\nOriginal Document");
        System.out.println(paragraph);
        System.out.println("\nEditing History");
        System.out.println(history);
        System.out.println("Final Document");
        System.out.println(finalDocument);
    }
}
```

INPUT

```
Java is easy? Java is powerful! I like Java.
Java
Python
```

OUTPUT

(no output)

Q2. Hospital Token Management System using List Interface**CODE**

```
import java.util.*;

public class HospitalTokenManagement {
    static List<Patient> patients = new LinkedList<>();
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        while (true) {
            int ch = sc.nextInt();
            switch (ch) {
                case 1:
                    registerPatient(sc);
                    break;
                case 2:
                    displayPatients();
                    break;
                case 3:
                    updatePatient(sc);
                    break;
                case 4:
                    removePatient(sc);
                    break;
                case 5:
                    displayDepartment(sc);
                    break;
                case 6:
                    firstLastPatient();
                    break;
                case 7:
                    System.out.println("Thank You");
                    return;
                default:
                    System.out.println("Invalid Choice");
            }
        }
    }

    static void registerPatient(Scanner sc) {

        int id = sc.nextInt();
        sc.nextLine();
        String name = sc.nextLine();
        String dept = sc.nextLine();
        patients.add(new Patient(id, name, dept));
        System.out.println("Patient Registered");
    }

    static void displayPatients() {
        if (patients.isEmpty()) {
            System.out.println("No Patients");
            return;
        }
        for (Patient p : patients)
            System.out.println(p);
    }

    static void updatePatient(Scanner sc) {
        int id = sc.nextInt();
        sc.nextLine();
        for (Patient p : patients) {
            if (p.id == id) {
                p.name = sc.nextLine();
                p.department = sc.nextLine();
                System.out.println("Patient Updated");
                return;
            }
        }
    }
}
```

Report

```

    }
    System.out.println("Patient Not Found");
}
static void removePatient(Scanner sc) {
    int id = sc.nextInt();
    Iterator<Patient> it = patients.iterator();
    while (it.hasNext()) {
        Patient p = it.next();
        if (p.id == id) {
            it.remove();
            System.out.println("Appointment Cancelled");
            return;
        }
    }
    System.out.println("Patient Not Found");
}
static void displayDepartment(Scanner sc) {
    sc.nextLine();
    String dept = sc.nextLine();
    boolean found = false;
    for (Patient p : patients) {
        if (p.department.equalsIgnoreCase(dept)) {
            System.out.println(p);
            found = true;
        }
    }
    if (!found)
        System.out.println("No Patients in this Department");
}
static void firstLastPatient() {
    if (patients.isEmpty()) {
        System.out.println("No Patients");
        return;
    }
    System.out.println("First Registered:");
    System.out.println(patients.get(0));
    System.out.println("Last Registered:");
    System.out.println(patients.get(patients.size() - 1));
}
}
class Patient {
    int id;
    String name;
    String department;
    Patient(int id, String name, String department) {
        this.id = id;
        this.name = name;
        this.department = department;
    }
    public String toString() {
        return "ID: " + id + " Name: " + name + " Department: " + department;
    }
}
}

```

INPUT

```

1
101
Rahul
Cardiology

```

```

1
102
Priya
Neurology

```

```

1
103
Kiran

```

Report

Orthopedics

2

3

102

Priya Sharma

Pediatrics

5

Pediatrics

6

4

101

2

7

OUTPUT

(no output)

Q3. Airport Boarding Queue Management using Queue Interface**CODE**

```
import java.util.*;

public class AirportBoardingQueue {

    static Queue<String> queue = new LinkedList<>();    static
    PriorityQueue<String> vipQueue = new PriorityQueue<>();

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        boolean run = true;

        while (run) {
            int ch = sc.nextInt();

            switch (ch) {

                case 1:
                    addPassenger(sc);
                    break;

                case 2:
                    displayQueue();
                    break;

                case 3:
                    boardPassenger();
                    break;

                case 4:
                    nextPassenger();
                    break;

                case 5:
                    addVIP(sc);
                    break;

                case 6:
                    System.out.println("Thank
You");
                    run = false;
                    break;

                default:
                    System.out.println("Invalid Choice");

            }
        }

        sc.close();
    }

    static void addPassenger(Scanner sc) {
        System.out.print("Enter Passenger Name:
");
        String name = sc.next();
        queue.offer(name);
        System.out.println("Passenger Added");
    }

    static void displayQueue() {
        System.out.println("VIP Queue: " + vipQueue);
    }
}
```

Report

```
        System.out.println("Normal Queue: " + queue);
    }
```

```
static void boardPassenger() {

    if (!vipQueue.isEmpty()) {
        System.out.println("Boarded VIP Passenger: " + vipQueue.poll());
    } else if (!queue.isEmpty()) {
        System.out.println("Boarded Passenger: " + queue.poll());
    } else {
        System.out.println("Queue Empty");
        return;
    }

    System.out.println("Remaining VIP Queue: " + vipQueue);
    System.out.println("Remaining Normal Queue: " + queue);
}

static void nextPassenger() {

    if (!vipQueue.isEmpty()) {
        System.out.println("Next VIP Passenger: " + vipQueue.peek());
    } else if (!queue.isEmpty()) {
        System.out.println("Next Passenger: " + queue.peek());
    } else {
        System.out.println("Queue Empty");
    }
}

static void addVIP(Scanner sc) {
    System.out.print("Enter VIP Passenger Name: ");
    String name = sc.next();
    vipQueue.offer(name);
    System.out.println("VIP Passenger Added");
}
}
```

INPUT

```
1
Rahul
1
Priya
5
Arjun
2
4
3
3
2
6
```

OUTPUT

(no output)

Q4. Digital Certificate Verification using Set Interface**CODE**

```
import java.util.*;

public class DigitalCertificateVerification {

    static LinkedHashSet<String> certificates = new LinkedHashSet<>();

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        boolean run = true;

        while (run) {
            int ch = sc.nextInt();

            switch (ch) {

                case 1:
                    addCertificate(sc);
                    break;

                case 2:
                    verifyCertificate(sc);
                    break;

                case 3:
                    displayInsertionOrder();
                    break;

                case 4:
                    displaySortedOrder();
                    break;

                case 5:
                    totalCertificates();
                    break;

                case 6:
                    removeCertificate(sc);
                    break;

                case 7:
                    System.out.println("Thank You");
                    break;

                default:
                    System.out.println("Invalid Choice");
            }
        }

        sc.close();
    }

    static void addCertificate(Scanner sc) {
        System.out.print("Enter Certificate ID: ");
        String id = sc.next();

        if (certificates.add(id))
            System.out.println("Certificate Added");
        else
            System.out.println("Duplicate Certificate Not Allowed");
    }
}
```

```
}

static void verifyCertificate(Scanner sc) {
    System.out.print("Enter Certificate ID: ");
    String id = sc.next();

    if (certificates.contains(id))
        System.out.println("Certificate Exists");
    else
        System.out.println("Certificate Not Found");
}

static void displayInsertionOrder() {
    System.out.println("Certificates:");
    for (String id : certificates)
        System.out.println(id);
}

static void displaySortedOrder() {
    TreeSet<String> sorted = new TreeSet<>(certificates);

    System.out.println("Sorted Certificates:");
    for (String id : sorted)
        System.out.println(id);
}

static void totalCertificates() {
    System.out.println("Total Unique Certificates: " + certificates.size());
}

static void removeCertificate(Scanner sc) {
    System.out.print("Enter Certificate ID: ");
    String id = sc.next();

    if (certificates.remove(id))
        System.out.println("Certificate Removed");
    else
        System.out.println("Certificate Not Found");
}
}
```

INPUT

```
1
C101
1
C102
1
C103
1
C101
2
C102
3
4
5
6
C102
3
7
```

OUTPUT

(no output)

Q5. Smart Inventory Management using Map Interface**CODE**

```
import java.util.*;

public class SmartInventoryManagement {

    static HashMap<Integer, Product> inventory = new HashMap<>();

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        boolean run = true;

        while (run) {
            int ch = sc.nextInt();

            switch (ch) {

                case 1:
                    addProduct(sc);
                    break;

                case 2:
                    updateQuantity(sc);
                    break;

                case 3:
                    searchProduct(sc);
                    break;

                case 4:
                    totalValue();
                    break;

                case 5:
                    displayCategoryWise();
                    break;

                case 6:
                    removeProduct(sc);
                    break;

                case 7:
                    displayProducts();
                    break;

                case 8:
                    System.out.println("Thank You");
                    break;

                default:
                    System.out.println("Invalid Choice");
            }
        }

        sc.close();
    }

    static void addProduct(Scanner sc) {
        int id = sc.nextInt();        String
        name = sc.next();
        String category = sc.next();
    }
}
```

Report

```
int qty = sc.nextInt();
double price = sc.nextDouble();

inventory.put(id, new Product(id, name, category, qty, price));

System.out.println("\nProduct Added");
}

static void updateQuantity(Scanner sc) {
int id = sc.nextInt();

    if (inventory.containsKey(id)) {            int
qty = sc.nextInt();
inventory.get(id).quantity = qty;
System.out.println("\nQuantity Updated");
    } else {
        System.out.println("\nProduct Not Found");
    }
}

static void searchProduct(Scanner sc) {
int id = sc.nextInt();

    if (inventory.containsKey(id))
System.out.println(inventory.get(id));        else
        System.out.println("\nProduct Not Found");
    }

static void totalValue() {

    double total = 0;

    for (Product p : inventory.values())
total += p.quantity * p.price;

    System.out.println("\nTotal Inventory Value: " + total);
}

static void displayCategoryWise() {

    for (Product p : inventory.values())
        System.out.println("\nCategory: " + p.category + " -> " + p);
}

static void removeProduct(Scanner sc) {

    System.out.print("\nEnter Product ID: ");
int id = sc.nextInt();

    if (inventory.remove(id) != null)
        System.out.println("\nProduct Removed");
else
        System.out.println("\nProduct Not Found");
}

static void displayProducts() {

    if (inventory.isEmpty()) {
System.out.println("\nNo Products");
return;    }

    for (Product p : inventory.values())
```

Report

```
        System.out.println(p);
    }
}

class Product {

    int id;
    String name;
    String category;
    int quantity;
    double price;

    Product(int id, String name, String category, int quantity, double price) {
        this.id = id;
        this.name = name;
        this.category = category;
        this.quantity = quantity;
        this.price = price;
    }

    public String toString() {
        return "ID: " + id + "\n"+
            " Name: " + name + "\n"+
            " Category: " + category + "\n"+
            " Quantity: " + quantity + "\n"+
            " Price: " + price;
    }
}
```

INPUT

```
1
101
Laptop
Electronics
5
50000
1
102
Mouse
Electronics
10
500
7
3
101
2
101
8
4
5
6
102
7
8
```

OUTPUT

(no output)